דף סיכום בחינה

מזהה סטודנט: [redacted]

מזהה קורס: 20594 שם קורס: מערכות הפעלה

מספר שאלה	ניקוד מירבי	ציון
1	25.00	18.00
2	17.00	16.00
3	13.00	11.00
4	5.00	4.00
5	5.00	5.00
6	5.00	4.00
7	5.00	5.00
8	5.00	1.00
9	5.00	5.00
10	5.00	5.00
11	5.00	5.00
12	5.00	5.00

ציון בחינה סופי : 84.00

הבחינה הבדוקה בעמודים הבאים

[redacted]

[redacted]

האוניברסיטה הפתוחה

ח' בתמוז תשפ"ב

מס' שאלון - 525

7
ביולי 2022

סמסטר 2022ב

מס' מועד 89

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.

את התשובות לכל השאלות יש לכתוב בשאלון הבחינה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלקים א׳ ו ב׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות**, **בכתב קריא ובקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך **אין לחרוג מהמקום המוקצה.**
2. בחלק ג׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה **יחידה הנכונה ביותר** מבין התשובות.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (55 נקודות)

שאלה 1 (25 נקודות)

מחשב חדש שנמצא בשלבי פיתוח מבוסס מעבד תומך בשתי פקודות מכונה מיוחדות לתיאום בין תהליכונים(THREADS). הקומפיילר תומך בפקודות המכונה הללו באמצעות מקרואים מיוחדים שמתורגמים לפקודות המכונה.

פקודת מכונה אחת מפעילים בעזרת המקרו m שהוא שקול לאופרטור * על מצביעים, כלומר v=m(p) שקול לטעינת הערך ששמור בכתובת p לתוך המשתנה v.

המקרו השני נקרא st והוא מיוחד יותר. הביטוי $r = st(v,p)$ מנסה לכתוב את הערך v לכתובת p בזיכרון.

הכתיבה מצליחה ומוחזר הערך 1, אם ורק אם התהליכון שמבצע את $st(v,p)$ ביצע לפני כן m(p) על הכתובת p וזו הפעולה האחרונה שבוצעה על הכתובת p ואף תהליכון אחר לא כתב ערך לכתובת p מאז שהתהליכון הזה ביצע m(p).

בכל מקרה אחר, כלומר אם התהליכון לא ביצע את m או אם תהליכון אחר כתב לכתובת p מאז שהוא ביצע את m (בין אם בעזרת st ובין אם בעזרת השמה בכל דרך), הכתיבה נכשלת (הערך v לא נכתב לכתובת p) והמקרו מחזיר את הערך 0 . במילים אחרות, הכתיבה מצליחה רק אם הערך שכתוב בזיכרון בעת ניסיון הכתיבה הוא אותו הערך שהתהליכון בעצמו קרא בעזרת ה m ואף תהליכון אחר לא כתב לתוכו מאז.

1 הצלחה
0 נכשל

(המשך השאלה בעמוד הבא)

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

א. **(15 נק')** ממשו שגרות lock(p) ו unlock(p) למניעה הדדית, כאשר p הוא מצביע לשלם שמייצג את המנעול. המימוש צריך להשתמש במקרואים(MACROS) המתוארים בשאלה.

```
int lock(p) {
m(p)
while (st(1, p) != 1) {
    m(p); \\ tries to "catch" p again
}
return 0; }
```

חסרה בדיקה שלא נעול

```
int unlock(p) {
    st(0, p);
    return 0;
}
```

אם לא מצליח?

הפונקציות מחזירות 0 בעת הצלחה. בפונקציית lock כל עוד לא מצליחים לכתוב 1 לתוך p (1 מסמל נעול כמו ב TSL) התהליך ינסה שוב "לתפוס" כשהוא מצליח סימן שהכתיבה הצליחה והתהליך "מחזיק" את p. ב unlock בהנחה שהתהליך נעל את p, לא אמורה להיות לו בעיה "לשחרר" את p ולכתוב לתוכו 0.

ב. **(10 נק')** מוגדר מבנה נתונים של רשימה מקושרת של מבנים מהטיפוס:

struct item { int data; struct item *next }

עם מצביע struct item *head לראש הרשימה. ממשו שגרה insert(int d) שמכניסה איבר חדש לרשימה שערך השדה data בו הוא d, וזאת תוך שימוש במקרואים m ו st. אסור להשתמש במנעולים ואסור להגדיר משתנים נוספים (פרט למשתנים אוטומטיים בשגרה).

השגרה צריכה לפעול נכון גם בסביבה שבה יש עוד תהליכונים שמנסים לפעול על הרשימה במקביל.

```
struct item *cur = head;
while (cur->next != null) {
    cur = cur->next
}
v = m(d);
Struct item new {int data; null};
r = st(v, data);
if (r=1) { cur->next = &new;}
else {return -1;} \\ insert failed
return 0; \\ insert succeeded
```

✓

לפי המנגנון למרות שהוא לא נכון

בהתחלה צריך להגיע לסוף הרשימה המקושרת וזה לא מצריך עדכון/נעילת משתנים. מנסים "לתפוס" את d בעזרת v=m(d). יוצרים struct חדש כדי להוסיף. אם תהליך אחר לא ניסה "לתפוס" את d בינתיים נוכל לכתוב את הערך של d ל struct החדש. אם הצלחנו נחבר את ה struct החדש לסוף הרשימה המקושרת ואם לא נחזיר שההכנסק נכשלה.

18
(1)

נא לא לכתוב בשוליים

89.31.5 M2 שאלון 525

נא לא לכתוב בשולים

שאלה 2 (17 נקודות)

מערכת מחשב מסוימת משתמשת בדפים בגודל 4KB ודיסק **קשיח** לדפדוף שקצב העברת הנתונים שלו הוא 10MByte/sec וזמן חיפוש(SEEK) ממוצע בו הוא 10ms(10 אלפיות שנייה). כל המרחב הווירטואלי של כל תהליך שמור בדיסק בצורה רציפה. הניחו שבכל שגיאת דף(PAGE FAULT) קיימות 2 אפשרויות בלבד: המערכת תביא דף בודד או 10 דפים רצופים(כמו שנאמר הם שמורים בדיסק ברצף).

הערה: בחישוב לא צריך לקחת בחשבון את הזמנים של פעולות טיפול אחרות בשגיאת דף (PAGE FAULT) שלגביהם אין נתונים בשאלה.

א. **(4 נק')**

האם מנגנון הבאת דפים מראש(PREPAGING) עשוי להאט תכניות לעומת המערכת דפדוף לפי דרישה(ON DEMAND PAGING)? נמקו. אם עניתם שכן, מהי ההאטה המרבית(במה ואיך היא תלויה)? תכתבו תיאור מילולי בלי חישובים.

תשובה והסבר:

כן. prepaging עלולה להאט ~~...~~ תוכנית מפויימת את הדפים ~~...~~ שהתהליך יצטרך כבר נמצאים. ~~...~~ למשל אם התהליך יכול להחליק רק 10 דפים אבל הוא משתמש רק בהם, prepaging ~~...~~ כנראה יחליף את הדפים האלה, יגרום ל page faults נוספים בהמשך, ויאט את התוכנית.

ב. **(7 נק')**

תארו מנגנון פשוט לקבלת החלטה ע"י מערכת הפעלה בזמן טיפול בשגיאת דף האם להביא דף בודד או 10 דפים. תארו את מבני הנתונים הדרושים למנגנון ואיזה גורם מעדכן את הנתונים האלה. המנגנון צריך לצרוך לא יותר ממספר קטן וקבוע של בתים בזיכרון לכל תהליך ולא יותר ממספר קטן וקבוע של פעולות חישוב בכל שגיאת דף. המנגנון חייב לקבל בד"כ החלטה נכונה במצבים שבם מתבצעות גישות אקראיות לזיכרון ראשי(למשל גישות לטבלת HASH) ובמצבים שבהם מתבצעות גישה רציפה לזיכרון (למשל חיפוש לינארי בטבלה גדולה). **מותר למנגנון לקבל החלטות שגויות במקרים אחרים.**

תשובה והסבר:

נשמור לתהליך משתנה num-pf זה שמתחל ל-0. הוא מסמל כמה פסיקות דף היו ברצף, כלומר כל פעם שתהיה פסיקת דף נעלה את המשתנה ב-1. בכל פעם שיש פסיקת דף נבדוק אם 6≤num-pf אם כן סימן שאולי להביא 10 דפים יהיה יותר משתלם, כי עשינו יותר מחצי מ-10 פסיקות דף, וניתן שהדפים שמורים בזכרון ברצף אז אולי זה משתלם לפי עקרון הלוקליות.

לא תמיד כדאי 10 חסר חישוב

אם 6≤num-pf נביא 10 דפים ונאפס את num-pf. אחרת, נביא רק דף אחד.

(המשך השאלה בעמוד הבא)

נא לא לכתוב בשוליים

שאלון 525 89.31.5 M2

נא לא לכתוב בשוליים

ג. **(6 נק')**

האם המנגנון שתיארתם בסעיף א' פועל בצורה נכונה כאשר התכנית מחשבת מכפלה פנימית של שני ווקטורים ארוכים? (כלומר עבור הלולאה `for ( i=0; i<N; i++ ) s+=A[ i ] * B[ i ];`).
הוכיחו שכן או הסבירו למה לא ותארו כיצד ניתן לשפר את המנגנון כך שיפעל נכון גם במקרה הזה.

תשובה והסבר:

לא. מכיוון שבטוח נרצה "להתקדם" בזכרון כדי לחשב את המכפלה הפנימית, לא נשתמש באותו דף פעמיים, מכיוון שגודל דף הוא 4K סביר להניח שלא נצטרך יותר מידי דפים (בהנחה שהווקטורים שמורים בצורה רציפה כפי שנתון בשאלה). לכן המנגנון בסעיף ב לא יביא ישר 10 דפים כמו שהיינו רוצים. כלומר האלגוריתם בסעיף ב' לא יעבוד בצורה נכונה במקרה זה.
ניתן לשפר את המנגנון ע"י הורדת הדגל של num-pf או מביאים 10 דפים מהזכרון.

16
(2)

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

שאלה 3 (13 נקודות)

כידוע, אחת הבעיות הפוגעות בביצועי מערכת הקבצים היא קבצים שאינם שמורים באופן רציף על הדיסק הקשיח.

א. **(3 נק׳)**

נציע שיטה חדשה להקצאת מקום לקובץ - ״שיטת הרווחים״. השיטה תשמש לצמצום ממדי הבעיה ע״י השארת מקום ריק (רווח) בגודל קבוע לאחר כל קובץ, כך שאם הקובץ צריך לגדול, הוא יכול לגדול באופן רציף. המקום השמור לאחר כל קובץ מיועד לקבוץ עצמו, אבל מותר להשתמש במקום זה עבור הקצאה לקבצים אחרים אם אין ברירה אחרת (כלומר רק במקרה שאין יותר מקום פנוי על הדיסק חוץ ממקומות השמורים לגדילה של קבצים).

תנו דוגמה פשוטה למקרה בו, כשמשתמשים ב״שיטת הרווחים״ אחד הקבצים אינו שמור באופן רציף.

תשובה והסבר:

שמרנו קובץ מסויים והשארנו לו מקום נוסף, אבל הוא גדל ביותר מגודל הרווח שהשארנו לו. לכן נצטרך להעביר את כולו למקום אחר כדי שיהיה שמור באופן רציף. אבל אין מקום מספיק גדול כדי לשמור אותו באופן רציף, אז נשמור אותו באופן לא רציף.

ב. **(5 נק׳)**

הועלתה הצעה הבאה (״שיטת החצי״): הקובץ הראשון ישמר בתחילת הדיסק. בהמשך, בכל פעם שנוצר קובץ חדש, נאתר את השטח הפנוי הרצוף הגדול ביותר על הדיסק, ונשמור את הקובץ החדש החל מאמצע שטח זה.

הסבירו מה היתרון לשמירת קובץ חדש באמצע הקטע הפנוי הרצוף הגדול ביותר ולא בתחילתו? האם שיטה זו תמנע לחלוטין מצב בו קובץ שמור בכמה חלקים נפרדים (כלומר, האם כל הקבצים יהיו שמורים ברצף)? נמקו.

תשובה והסבר: שנגדיל את הסיכוי

היתרון הוא שכשנמחק את הקובץ נצרף את השטח הפנוי החדש לשטח פנוי נוסף, וככה יהיה לנו יותר שטח פנוי ברצף ונקטין את הריסוק החיצוני. השיטה לא תמנע לחלוטין מצב שבו קובץ שמור בכמה חלקים נפרדים, למשל במקרה שהזכרון יהפוך מלא והקבצים נמחקים ממקומות שונים בזכרון ולא מאותו אזור.

(המשך השאלה בעמוד הבא)

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

ג. **(5 נק')**

נניח שאנו משתמשים ב"שיטת החצי" כדי למקם קבצים חדשים. מבין השיטות למימוש קבצים שלמדנו, איזו המתאימה ביותר לצרכנו? השיטות שלמדנו הן מימוש קבצים באמצעות הקצאה רציפה (contiguous allocation), מימוש קבצים באמצעות הקצאה בשיטת רשימה מקושרת (linked list allocation), מימוש קבצים ברשימה מקושרת באמצעות אינדקס (מבוססת i-nodes)?. נמקו.

תשובה והסבר:

עדיף להשתמש ברשימה מקושרת. בשיטה זו אפשר להחזיק 2 רשימות מקושרות, אחת לבלוקים של זכרון פנוי ואחת לבלוקים של זכרון תפוס. אפשר לשמור את הבלוקים הפנויים לפי גודל המקום, ואז ~~[illegible]~~ חיפוש השטח הפנוי הגדול ביותר לה יהיה מהיר. כשנרצה לשמור את הקובץ החדש נרשום את המקום כתפוס ברשימה המקושרת של הבלוקים הפנויים, ונפצל את ה-node שמתאר את המקום הפנוי ל-2.

11
(3)

השאלה לא על מעקב מקום פנוי

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

חלק ב (25 נקודות)

לפניכם חמש שאלות. יש לענות על כולן.

5 נקודות לשאלה ; 25 נקודות לחלק זה.

שאלה 4 (5 נקודות)

הסבירו מה תהיה המשמעות אם במערכת XV6 שתומכת בקונטיינרים לא בוצע בכלל כתיבת "+cpu" לקובץ cgroup.subtree_control : קונטיינר לא ייווצר, ייווצר, אך לא יתפקד בכלל כי לא יקבל את זמן CPU או ייווצר, אך לא יתפקד כמצופה ?

4
(4)

תשובה והסבר :

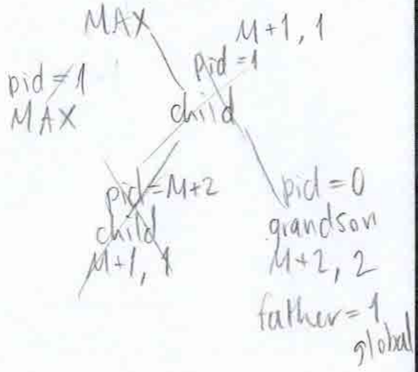
הקונטיינר ייווצר, אך אם לא בוצע כתיבה "+cpu" לא יתווספו אופציות כמו cpu.max וcpu.weight לcontroller. לכן הקונטיינר ייווצר אך לא יתפקד כמצופה. חסר פירוט במה?

שאלה 5 (5 נקודות)

במערכת XV6 קיימת קריאת מערכת KILL (PID) שמאפשרת להרוג את התהליך ע"פ PID שלו. פונקציה getppid() מחזירה את ה PID של האב(FATHER), ל PROCESS עם PID=1 , היא מחזירה 0.

מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?

```
unshare(PID_NS);
int pid = fork();
if(pid == 0) {
  pid = fork();
  if(pid == 0) {
    sleep(999); // long sleep
    kill(getppid());
    printf(stdout, "GrandSon PID=%d \n", getpid());
  }
  else {
    sleep(9999); // very long sleep
printf(stdout, "Son PID=%d \n", getpid());
  }
}
else {
  sleep(99999); //very very sleep enough for creating and running all son processes
  printf(stdout, "Father still alive \n");
}
```



נא לא לכתוב בשוליים
שאלון 525
89.31.5 M2

נא לא לכתוב בשוליים

ניתן להניח שכל קריאות מערכת מצליחות ושלא נוצרו תהליכים נוספים במערכת מרגע הרצת הקוד(חוץ מאלה שהקוד יוצר) ושמערכת מקצה PID עוקב כל פעם שנוצר תהליך חדש.

תשובה והסבר:

יודפס GrandSon PID=2 ה unshare מאתחל את המספור

~~Son PID = 1~~ של ה pid namespace החדש שנוצר

Father Still alive עם fork. הנכד מקבל pid הולך לישון

להכי פחות לזמן, קם ועושה הורג את אביו

שלו עם pid=1. אחרי זה מודפס הpid של האבא (בן של הסבא) ולבסוף

הסבא מדפיס father still alive

5
(5)

שאלה 6 (5 נקודות)

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
int pid = fork();
if (pid == 0) {
  mkdir ("dirS");
  mount ("internal_fs_a","dirS",0); //mount a preformatted internal_fs_a
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
  }
if (pid > 0) {
sleep (10000); // wait for child to finish
unshare (MOUNT_NS);
  }
sleep (10000); // both processes will sleep
if( pid == 0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to open\n");
   else printf (stderr, "son succeeded\n");
  }
If (pid>0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
 }
```

(הערות בכתב יד ליד הקוד: בן; הבן כותב לקובץ; אבא; האבא מתנתק; סנכרון; בן; אבא)

תשובה והסבר:

יודפס son succeeded ובשורה נפרדת father succeeded

כי ב unshare האבא מתנתק ~~ביחד עם~~ הבן שלו, ולכן

לשניהם יש את אותה הגישה לקבצים, כולל ל dirS/file.txt

4
(6)

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

שאלה 7 (5 נקודות)

לדיסק בעל 40 גלילים(CYLINDERS), כאשר הזרוע נמצאת בגליל 13 (בכיוון תנועה ושירות בקשות כלפי מעלה- הגליל בעל המספר הגבוה ביותר), מגיעה סדרת בקשות קלט/פלט לגלילים הבאים (משמאל לימין):

22, 11, 36, 15, 39, 1

איזה מאלגוריתמי תזמון הדיסק הבאים SSF - החיפוש הקצר קודם , SCAN - אלגוריתם המעלית
ו C-SCAN - סריקה מחזורית בכיוון למעלה בלבד יתבצע תוך השגת מינימום מרחק חיפוש כוללי? הראו את החישובים בבירור.

5
(7)

חישוב והסבר:

SSF: 13 →(2) 15 →(4) 11 →(10) 1 →(21) 22 →(14) 36 →(3) 39 = 54

C-SCAN: 13 →(2) 15 →(7) 22 →(14) 36 →(3) 39 →(38) 1 →(10) 11 = 74

SCAN: 13 →(2) 15 →(7) 22 →(14) 36 →(3) 39 →(28) 11 →(10) 1 = 64

C-SCAN יתבצע תוך השגת מינימום מרחק חיפוש.

★ הנחתי שיותר נכון לעשות 13→15→11 לפי כיוון תנועה למעלה, אבל למעשה 13→11→15 היה מביא את אותה תוצאה.

שאלה 8 (5 נקודות)

איזו תכונה חשובה נדרשת למעבד בשביל שיהיה ניתן לממש עליו את ביצועי הווירטואליזציה בשיטת Trap-and-Simulate?

1
(8)

לא במעבד

Hypervisor. כי התהליך/תוכנה שמופעלת [crossed out] בוירטואליזציה ורוצה לפעול במצב ראשוני עושה trap, מעבירה את הבקשה ל hypervisor והוא [crossed out] מחקה את פעולת ה Kernel ואז מחזיר את השליטה לתהליך/תוכנה.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

שאלון 525 89.31.5 M2

נא לא לכתוב בשולים

חלק ג (20 נקודות)

לפניכם 4 שאלות. יש לענות על כולן.

(5 נקודות לשאלה ; 20 נקודות לחלק זה).

שאלה 9 (5 נקודות)

אחד ההבדלים בין פונקציות ספריה בשפת C (כגון printf) לבין קריאות מערכת (system calls) ב- Linux הוא :

א. רק למנהל המערכת (super user) מותר להשתמש בקריאות מערכת, בפונקציות ספריה יכול להשתמש כל אחד

ב. **כל** קריאות המערכת הן חלק מפונקציות הספרייה של שפת C

ג. **כל** פונקציות הספרייה ממומשות בעזרת קריאות מערכת

ד. כל התשובות הקודמות אינן נכונות

5 (9)

שאלה 10 (5 נקודות)

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node.

- גודל הבלוק במערכת הקבצים הוא 1 KBytes
- כתובת הבלוק היא 4 בתים (bytes)
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

חשבו מהו הגודל המקסימלי של קובץ במערכת הקבצים הזו?

הערה : לצורך החישוב 1 GByte = 1024 * 1024 Kbyte , 1 Kbyte = 1024 Byte

5 (10)

א. 15 GBytes

ב. 15.5 GBytes

ג. 16 GBytes

ד. 16.5 GBytes

$\frac{1024 \text{ bytes}}{4 \text{ bytes}} = 256$ מצביעים מכל בלוק

b גודל בלוק

$b(10+256+256^2+256^3)$

$= 16843018$ KB

16 GB

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

נא לא לכתוב בשולים

שאלה 11 (5 נקודות)

מהו החיסרון המובהק של שיטת ה- rendezvous לעומת ה- mailbox ב- message passing?

5 (11)

א. שיטת ה-rendezvous קשה יותר למימוש במערכת הפעלה.

ב. שיטת ה-rendezvous פחות גמישה מכיוון שהתהליכים (השולח והנמען) הופכים להיות מסונכרנים (תלויים זה בזה).

ג. בשיטת ה-rendezvous לא ניתן לממש מניעה הדדית בגישה לקטעים קריטיים (critical sections).

ד. שיטת ה-rendezvous גורמת לעומס קיצוני על מנגנון התזמון(SHEDULING)

שאלה 12 (5 נקודות)

נתון כי במערכת מסוימת המשתמשת בשיטת הדפדוף (paging) לצורך ניהול הזיכרון :

- גודל הדף הוא 4 K.
- הכתובת הוירטואלית הנה בת 20 סיביות (bits).
- גודל הזיכרון הפיזי הוא 1024 KB = 1 MB.
- גודל שורה בטבלת הדפים (page table entry) הוא 4 בתים (bytes).

5 (12)

אם טבלת הדפים הנה טבלת הדפים המהופכת (inverted page table) מהו גודל הטבלה?

א. 2560 בתים.

ב. 512 בתים.

ג. 2048 בתים.

ד. 1024 בתים.

$\frac{1024\,KB}{4\cdot 2^{10}} = 256$

$256 \cdot 4 = 1024$

בהצלחה!

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים